

lab4

任务介绍

本次实验将实验3生成的中间代码翻译成目标riscv代码，本人选择的中间代码是Zero IR

功能实现

Lab4 主要完成从中间代码到 RISC-V 目标代码的生成过程，包含几个关键阶段。

1. 指令选择 (InstSelector)

指令选择器负责将 IR 指令转换为 RISC-V 汇编指令，主要功能：

关键实现点：

一对多转换：一条IR指令可能需要转换为多条RISC-V指令

处理立即数限制：RISC-V指令的立即数范围有限制，需特殊处理

数组处理：正确计算数组基址和偏移量

2. 寄存器分配 (RegAllocator)

由于硬件寄存器数量有限，需要将IR中的虚拟寄存器映射到物理寄存器或栈内存：

实现策略：

朴素寄存器分配：每条指令执行前从栈加载操作数，执行后将结果存回栈

变量栈偏移计算：为每个变量在栈上分配空间并记录偏移量

特殊处理数组和普通变量：通过 `alloc_temp` 的 `is_array` 参数区分

3. 栈帧管理

通过 `Function` 类管理栈帧分配：

关键实现：

数组变量：使用正偏移量，确保在栈帧内

普通变量：使用负偏移量，保持与原有逻辑兼容

栈指针调整：在函数入口处正确调整栈指针

4. 汇编代码生成 (ASMEmitter)

将优化后的汇编指令转换为文本形式的汇编代码：

主要功能：

函数序言和尾声：生成栈帧设置和恢复代码

全局变量处理：在数据段生成全局变量

汇编指令输出：按特定格式输出每条汇编指令

实现过程如下：

1. 将Zero IR的`x = #k`指令转换为RISC-V的 `li reg(x), k` 指令
2. 将Zero IR中的 `GOTO label` 指令翻译为RISC-V汇编中的 `j label` 指令
3. 将Zero IR中的 `LABEL label:` 标签定义指令翻译为RISC-V汇编中的 `label:` 标签定义
4. 将Zero IR中的二元运算指令翻译为RISC-V汇编代码
5. 将Zero IR中的单目运算指令翻译为RISC-V汇编代码
6. Zero IR的条件分支指令 `IF x relop y GOTO L` 到RISC-V的条件分支指令（如 `beq`, `bne`, `blt`, `bge` 等）
7. 将Zero IR的 `CALL f` 函数调用指令翻译为RISC-V的函数调用指令
8. `load,store`翻译
9. 实现函数调用参数传递，前8个放寄存器，超过8个入栈
10. 将Zero IR中的 `x = &label` 指令翻译为RISC-V的 `la reg(x), label` 指令的功能
11. 实现函数返回翻译

12. 实现栈帧管理功能
13. DEC x #k -> 在栈上分配k字节空间并将地址存入x
14. 完善selectLoad函数，以支持带偏移量的内存访问
15. 添加函数参数声明翻译功能
16. 添加立即数算数指令支持
17. 完善参数索引计数
18. 在ASMEmitter类中添加函数入口(prologue)和出口(epilogue)代码生成的能力
19. 实现寄存器分配功能
20. 添加全局变量声明翻译功能
21. 布局调整：修复函数声明和栈帧结构
22. 重构reg_allocate中的allocate函数，防止嵌套过深

间隔2个半月后

23. 拉取模版与测试更新
24. 重新测试lab1,lab2,lab3，看看能否通过所有的测试
25. 解决copilot不可用问题
26. 研究如何手动调试lab1

代码块

```
1  make clean
2  make
3  ./compile 测试文件相对路径
```

27. tests/lab1/empty_stmt.sy 重新编译后无错
28. 更新参考文档
29. 顺序解决前面实验留存的漏洞：合并后似乎代码出现了一些问题，lab1全对，但lab2,3报错严重，待查明，从lab2开始检查问题所在 --- 重新编译一下又好了
30. 回忆起lab2怎么debug --- ./compiler file_path（不通过的文件路径），看运行信息
31. 拉一下lab4的新模版，Merge一下，应该有改动 --- 没有改动
32. tests/lab1/empty_stmt.sy Compile Error ---- 已解决，原因是生成中间代码时缺少对EmptyStmt的识别，抛出了异常，已添加识别
33. tests/lab2/cond_with_arr.sy Compile Error --- 添加对EmptyStmt的识别后连带解决
34. 回忆一下lab3如何debug看运行输出情况: ./compiler file_path output_file_path --ir

- 35. 重构对多维数组初始化的支持成递归形式，可以支持任意维数数组（原来只能支持1-2维）
- 36. 再读一遍lab4指导书 -- 读完了
- 37. 解决lab4测试结果不匹配问题：单独测试输出正常，脚本测试无输出 --- .toml配置问题，最后一行注释掉，别的不注释即可
- 38. 修复寄存器分配：采用基于riscv代码的朴素寄存器分配
- 39. 修复数组访问时的存储位置计算错误

实现中的亮点

完成了对之前阶段代码的查漏补缺和优化，如将中间代码生成阶段处理数组初始化的代码重构成递归形式，使其能够支持任意维数组的初始化，而不是像之前的代码迭代式处理，只能实现两维以内的数组初始化。

AI工具与代码借鉴使用声明

本实验本人没有借鉴任何除统一提供的模版之外的代码。

本报告没有使用AI工具编写。

本实验的代码部分由本人进行构思和调试以及设计性编写，而部分结构重复的翻译工作由AI（github copilot）生成。

建议和想法

想法：由于个人时间安排原因（毕设+考研复习排期），lab4暂时不打算完整完成了，只能算是写得马马虎虎，可能之后有时间再慢慢完善吧。